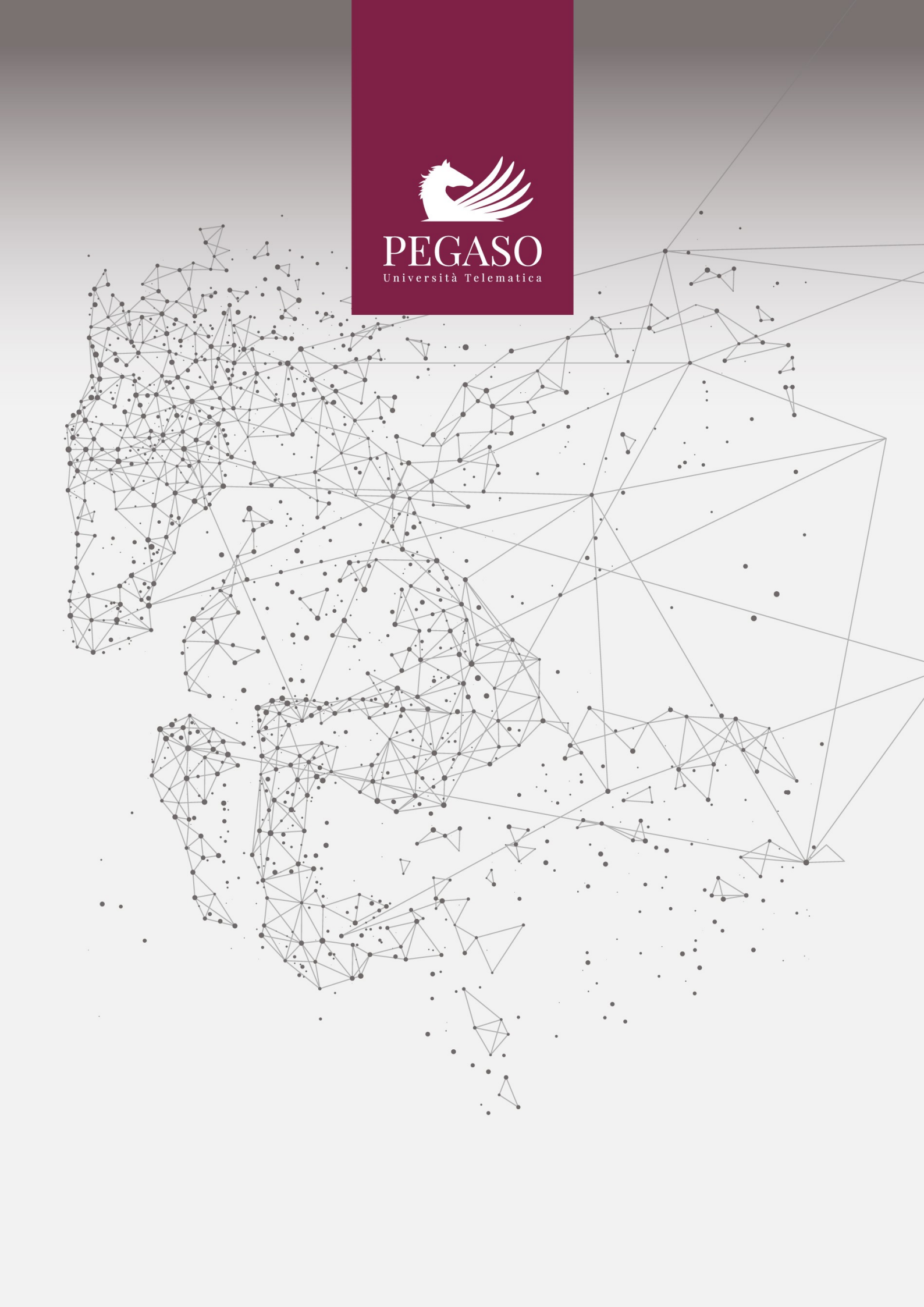

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. HTTP....3
2. RISORSE....5
3. METHOD....8
BIBLIOGRAFIA E SITOGRAFIA....11

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. HTTP

L'**HyperText Transfer Protocol** (**HTTP**) è un protocollo a livello applicativo usato come principale sistema per la trasmissione d'informazioni sul web ovvero in un'architettura tipica **Client-Server**. L'architettura su cui si innesta l'HTTP è di tipo **Client-Server**:

- il **client** invia una richiesta al server, specificando l'URL della risorsa che desidera recuperare;
- il **server** risponde con un messaggio di stato che indica se la richiesta è stata eseguita con successo o meno, e invia i dati richiesti al client.

Nell'uso comune il client corrisponde al browser ed il server la macchina su cui risiede il sito web. Vi sono quindi due tipi di messaggi HTTP: messaggi richiesta (detti **HTTP requests**) e messaggi risposta (detti **HTTP responses**).

Consideriamo una richiesta che facciamo dal nostro browser, per raggiungere la seguente url: http://www.mydestination.it:80/mypage.html

Gli steps che avvengono nel client sono i seguenti:

- Il browser estrae l'hostname: www.mydestination.it
- Il browser cerca l'IP associato all'hostname (tramite il DNS): supponiamo che l'IP sia 1.2.3.4
- Il browser estrae la porta: 80
- Il browser effettua una connessione TCP verso l'ip associato all'host, sulla porta 80: 1.2.3.4:80
- Il browser esegue una HTTP request di tipo GET al server
- Il browser legge l'HTTP response dal server
- Il browser chiude la connessione

Una transazione HTTP standard prevede, nell'ordine:

- Di instaurare una connessione TCP (client → server)
- Invio di un messaggio request (client → server)
- Invio di un messaggio response (client ← server)
- Chiusura della connessione HTTP

Se il contenuto dei messaggi è ridotto, l'overhead della connessione diventa significativo. Vi sono due possibili soluzioni:

1. Il client instaura più connessioni parallele verso il server
    a. Vantaggi: molto rapido, può gestire risorse su server diversi
    b. Svantaggi: elevato utilizzo di risorse del client (memoria), non porta vantaggi se la banda del client è limitata

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Il client riutilizza la stessa connessione TCP per molteplici messaggi sequenziali. Supportato in HTTP/1.1
    a. Vantaggi: rapido se si inviano molti messaggi di dimensione ridotta

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Risorse

Il server HTTP ospita (host) delle risorse. Le risorse possono essere statiche o dei programmi che generano contenuto on-demand.

Ciascuna risorsa ha un tipo, detto MIME (Multipurpose Internet Mail Extension). Il tipo descrive il formato della risorsa. Il tipo mime è semplicemente una stringa, formattata in questo modo: <tipo>/<sottotipo>

Esempi:

- text/html è il tipo mime per un documento HTML
- text/plain è il tipo mime per un semplice file di testo con codifica ASCII
- image/jpeg è il tipo mime per un'immagine JPEG
- application/JSON è il tipo mime per stringhe JSON
- etc.

Ciascuna risorsa ha un nome che la identifica in modo univoco sul web, detto **URI** (**Uniform Resource Identifier**).

Due differenti tipologie di URI: **URL** e **URN**

- Uniform Resource Locator identifica una risorsa in base al posto in cui si trova.
- Uniform Resourse Name serve come nome univoco per la risorsa, indipendentemente da dove si trova (non è usato nella pratica).

Facciamo un esempio di vita "reale": mentre un URN può essere paragonato al nome di una specifica persona, l'URL può essere pensato come l'indirizzo della residenza di quella persona. Quindi, mentre l'URN identifica univocamente la persona, l'URL fornisce un mezzo per trovarla.

Un Uniform Resource Locator (URL) è di fatto un nome standardizzato per definire risorse nel web:

- Solitamente rappresenta l'entry-point da cui gli utenti iniziano a navigare sul web.
- Fornisce una codifica del nome della risorsa, il luogo dove trovarla e il modo in cui è possibile ottenerla.

Il formato generale è composto da una stringa composta da 9 parti distinte (non tutte obbligatorie)

<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>

Ad es. http://www.miosito.it/percorso/index.html

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Analizziamo le singole parti di cui è composta una url:

- **<scheme>** → lo schema specifica che protocollo usare quando si accede al server per ottenere la risorsa indicata. (ad es. http, ftp, rtsp, etc.)
- **<user>:<password>** → L'utente e la password possono essere talvolta richiesti su alcuni protocolli per autenticarsi con il server. Se non indicato, l'utente di default è "anonymous"
- **<host>** → L'indirizzo del server da cui è possibile reperire la risorsa. Può essere un hostname oppure un indirizzo IP. Il valore deve essere sempre specificato
- **<port>** → La porta (TCP) su cui il server sta attendendo connessioni. A seconda dello schema specificato, la porta può avere un valore di default e può essere omessa. (per HTTP la porta di default è la 80)
- **<path>** → Il nome (locale) della risorsa sul server. La sintassi del path è specifica della combinazione schema-server. In HTTP il path può essere diviso in più segmenti separati da "/" (ad . folder1/folder2/folder3)
- **<params>** → Questa componente permette di associare liste di parametri nome-valore ad un segmento del path. Questo può essere necessario per fornire al server web ulteriori informazioni (parametri) sulla risorsa richiesta
- **<query>** → Simile a <params> (e più usato in ambito HTTP), <query> permette di definire coppie nome-valore per restringere il contesto della risorsa richiesta; le coppie chiave-valore possono essere separate dal carattere & (ad es. http://ww.miosito.it/pagina.html?id=1234&name=hello → in questo caso la risorsa pagina.html viene ristretta a riferirsi soltanto ad un ipotetico id numero 1234 e nome hello)
- **<frag>** → Fragment può essere usato per identificare frammenti all'interno della risorsa specificata. Per esempio, in un file HTML può puntare ad un paragrafo o ad un'immagine (visto che solitamente il server gestisce intere risorse, e non parte di esse, la parte di fragment non è inviata al server durante una richiesta HTTP. Essa viene usata dal client in fase di visualizzazione)

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

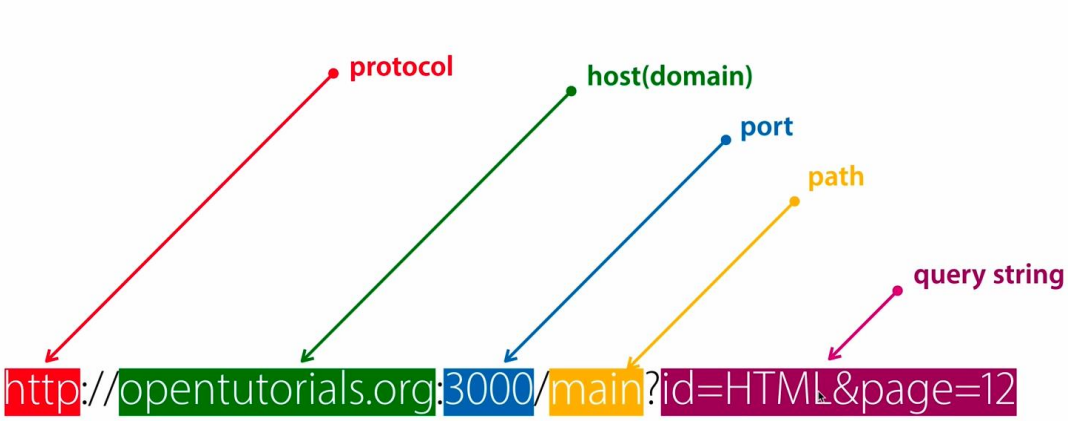


Figura 1 *- Esempio di URL*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Method

I messaggi di tipo HTTP request, inviati dal client al server per richiedere un'azione, sono composti in questo modo:

- **Start line:** <method> <request-URL> <version>
- **Header:** <name1>: <value1> <name2>: <value2> ...
- **Body:** Dati binari o un semplice CRLF se non sono presenti dati

Il <method> specificato nella start line può contenere i seguenti valori: **GET**, **HEAD**, **PUT**, **POST**, **TRACE**, **OPTIONS**, **DELETE**, […]

Analizziamo nel dettaglio le principali i 2 metodi GET e POST.

GET è utilizzato per richiedere informazioni da un server web. Le informazioni vengono inviate come parte dell'URL e possono essere facilmente salvate o condivise. Le richieste GET sono sicure, non modificano i dati sul server e possono essere facilmente memorizzate nella cronologia del browser.

POST è utilizzato per inviare informazioni al server per la creazione o l'aggiornamento di una risorsa. Le informazioni vengono inviate come parte del corpo della richiesta e non possono essere facilmente salvate o condivise. Le richieste POST sono meno sicure rispetto alle richieste GET poiché possono modificare i dati sul server e non possono essere memorizzate nella cronologia del browser.

Di seguito mostriamo una tabella "riassuntiva" delle differenze tra i 2 metodi:

GET	POST
È utilizzato per richiedere informazioni da un server web.	È utilizzato per inviare informazioni al server
È utilizzato per recuperare dati	È utilizzato per creare o aggiornare dati
È limitato per quanto riguarda la quantità di dati che è possibile inviare al server, poiché i dati sono inviati come parte dell'URL	Non ha questa limitazione poiché i dati vengono inviati come parte del corpo della richiesta
È idempotente, ovvero l'esecuzione ripetuta di una richiesta GET con gli stessi parametri non ha alcun effetto collaterale sul server o sui dati	Non è idempotente, poiché ogni richiesta POST può avere effetti collaterali diversi sul server e sui dati
È più sicuro poiché i dati inviati come parte dell'URL possono essere facilmente registrati	È meno sicuro perché i dati inviati come parte del corpo della richiesta non possono essere facilmente

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

nella cronologia del browser e nei registri del server	registrati.
È utilizzato per recuperare dati da un server in modo leggibile	È utilizzato per inviare dati al server in modo non leggibile
È utilizzato per richiedere pagine web e risorse statiche	È utilizzato per richiedere l'elaborazione di form o l'invio di dati sensibili, come le informazioni di accesso

Oltre all'utilizzo dell'**esamina** in EDGE e gli **strumenti per lo sviluppo** di Chrome, è possibile usare degli appositi tool per testare le varie chiamate HTTP e leggere agevolmente le risposte del server.

Tra questi riportiamo **POSTMAN** (Postman API Platform | Sign Up for Free)

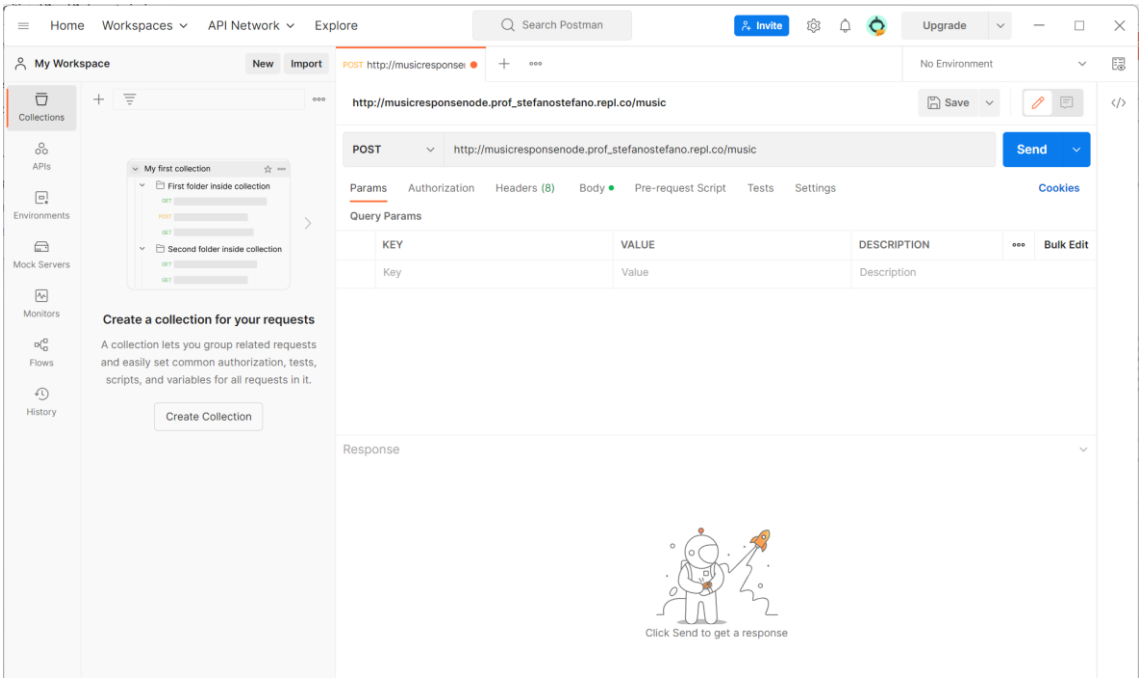


Figura 2 *- POSTMAN app*

Un'altra opzione è quella di installare add-on sul browser; ve ne sono svariati sui vari browser "store", segnaliamo uno tra i tanti:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

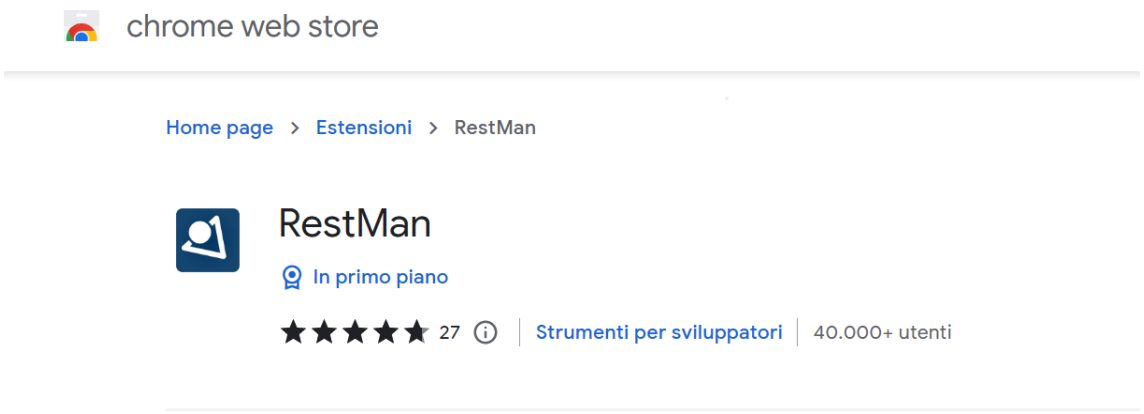


Figura 3 *- Browser add-on*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia e sitografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- F. Bergamasco - WEB APPLICATIONS AND TECHNOLOGIES - CT0142
- https://www.evemilano.com/protocolli-http/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).